

SQL Injection

Laboratory for the class “Offensive Security”
(01NWLWQ, 01NWLUV, 01NWL UW, 01NWLWR)
Politecnico di Torino – AY 2025/26
Prof. Cataldo Basile

prepared by:
Stefano Alberto (stefano.alberto@polito.it)

v. 1.0.0 (17/4/2026)

Contents

1	Introduction	2
2	Exercises	4
3	Extras	12

1 Introduction

The purpose of this laboratory is to understand and exploit SQL injection vulnerabilities. SQL injection is one of the most well-known and impactful classes of web vulnerabilities, and it serves as a concrete example of a broader category: **injection attacks**.

Injection vulnerabilities

An injection vulnerability arises whenever user-controlled input is passed to another component that **interprets** it as code or commands. Because the receiving component cannot distinguish between intended instructions and the attacker's input, the attacker can alter its behavior.

This is a broad category that includes SQL injection, OS command injection, LDAP injection, code injection, and more. The common root cause is always the same: mixing **data** (user input) with **code** (instructions for another component) in a single channel. In this lab, we focus on SQL injection, where the other component is a relational database.

SQL Injection

In a typical web application, the backend builds SQL queries to interact with the database. SQL injection occurs when user-controlled input is **concatenated directly** into the query string. The backend and the database are two separate components: once the SQL query is constructed and sent, the database has no way to tell which parts came from the developer and which parts came from the user.

The canonical vulnerable pattern looks like this:

```
query = "SELECT * FROM users WHERE username = '" + input + "'"
```

If the user provides a normal value such as `alice`, the resulting query is harmless:

```
SELECT * FROM users WHERE username = 'alice'
```

But if the user provides a crafted value such as `' OR 1=1 --`, the query becomes:

```
SELECT * FROM users WHERE username = '' OR 1=1 -- '
```

The single quote closes the string literal, `OR 1=1` makes the condition always true, and `--` comments out the rest of the query. The attacker has injected SQL code that changes the query's meaning entirely.

Because SQL is a rich and expressive language, there are many techniques to exploit an injection point depending on the context. The main ones, which we will explore in the exercises, are:

- **Logic-based:** the attacker alters the boolean logic of the query (e.g. turning a `WHERE` condition into a tautology). This is useful when the query itself controls a security-critical decision, such as authentication.
- **UNION-based:** the attacker appends a `UNION SELECT` to the original query, retrieving data from other tables. This is useful when the vulnerable query reads from an uninteresting table but displays its results on the page.
- **Error-based:** the attacker triggers database errors that include data from the database in the error message. This is useful when the application does not display query results directly but does show error messages.
- **Time-based (blind):** the attacker injects conditional time delays (e.g. `SLEEP()`) and infers data one bit at a time by measuring response times. This is the last resort when no other output channel is available.

Prevention: Prepared Statements

The correct defense against SQL injection is to use **prepared statements** (also called parameterized queries). Instead of concatenating user input into the query string, the developer writes a query template with placeholders and passes the user input separately:

```
query = "SELECT * FROM users WHERE username = ?"
```

The database driver sends the query structure and the parameter values through **separate channels**. The database knows exactly which part is code and which part is data, making injection impossible regardless of the input content.

NOTE

Prepared statements do not sanitize or escape the input; they eliminate the problem entirely by keeping code and data separate. This is fundamentally more robust than any filtering or escaping approach, which can always be bypassed by a sufficiently creative attacker.

For a comprehensive overview of SQL injection techniques and practice labs, see:

<https://portswigger.net/web-security/sql-injection>

2 Exercises

This section includes six exercises on SQL injection. The first two are solved step by step using the OliCyber SQLi tutorial series; the next three are standalone challenges with less hand-holding; the last one is optional and covers a more advanced scenario.

The OliCyber tutorial is a dedicated web application at <https://training.olicyber.it/challenges#challenge-356>. It has a navigation bar with four challenge pages (Logic SQLi, Union SQLi, Blind SQLi, Time-based SQLi). Each page has a left panel with step-by-step instructions and a right panel with a console where you type input and submit it. After submitting, the application shows the SQL query that was actually executed, the result (if the query succeeded), or the error message (if the query was invalid). The homepage also provides a Python helper class for scripting API requests.

Exercise 1 – SQLi Tutorial: Logic-based

Purpose:	Bypass a simulated login check using a logic-based SQL injection.
Suggested tools:	Browser (or the Python helper class from the tutorial homepage).
Source:	OliCyber Training – Web 17 - SQLi 1 – https://training.olicyber.it/challenges#challenge-356

This is the first challenge in the OliCyber SQLi tutorial series. The page simulates a password-based login: you type input into a console, and the backend inserts it into a SQL query that checks the `login` table. If the query returns at least one row, the login succeeds; otherwise, it fails. The challenge also shows contextual instructions on the left side of the page, read them as you go. Follow the steps below.

Explore the console

Open <http://web-17.challs.olicyber.it/logic> in your browser. On the right, you see a text area (“Console”) with a *Send* button. On the left, the tutorial text explains what to do.

Type a random string, e.g. `foobar`, and click *Invia* (Send). Observe the three pieces of feedback the application gives you:

1. **Query executed** — the exact SQL that was run.
2. **Result** — the query output (or `Fail`).
3. **Error** — shown only when the query has a syntax error.

What query was executed? What was the result?

→

Understand the query

The application concatenates your input directly into a `SELECT` query:

```
SELECT * FROM login WHERE password = 'your_input'
```

The backend is MySQL (PyMySQL/SQLAlchemy). Your input lands inside the single-quoted string literal. If the query returns at least one row, the result is the flag; otherwise the result is `Fail`.

Given what you learned in the introduction, what is the problem with this query?

→

Break the syntax

Before crafting a full payload, confirm that you can escape the string literal. Send the input `foo' bar` and observe what happens.

→

Craft a payload

Your goal is to make the `WHERE` condition always true so that the query returns all rows from the `login` table. You need to:

1. Close the string literal with a single quote `'`.
2. Append `OR 1=1` to create a tautology.
3. Comment out the rest of the original query (the trailing quote) with `-- -`.

Send the following input:

```
' OR 1=1 -- -
```

What does the resulting query look like? What is the result?

→

NOTE

The `--` sequence begins a single-line comment in SQL (MySQL requires at least one whitespace character after `--`). Everything after it on the same line is ignored by the database. This lets the attacker neutralize the remaining parts of the original query that would otherwise cause a syntax error. A common convention is `-- -` (dash dash space dash) to ensure the space is preserved even if the input is trimmed.

Reflection

This type of SQL injection is called **logic-based** (or **authentication bypass**) because it alters the query's boolean logic. It is effective when the vulnerable query directly controls a security-critical decision, such as granting access.

Think about its limitations: what if the vulnerable query is a `SELECT` that displays data from a table you are not interested in? Would a logic-based injection help you read data from other tables?

→

Exercise 2 – SQLi Tutorial: UNION-based

Purpose:	Extract data from a different table using a UNION-based SQL injection.
Suggested tools:	Browser (or the Python helper class from the tutorial homepage).
Source:	OliCyber Training – Web 17 - SQLi 2 – https://training.olicyber.it/challenges#challenge-356

This is the second challenge in the OliCyber SQLi tutorial series. The interface is the same as Exercise 1: a console on the right and step-by-step instructions on the left. This time, the backend query searches a table called `dummy_data` by `id`. The table contains only dummy values, with no flag. Your task is to discover where the flag is stored and extract it using a `UNION SELECT` query. Follow the steps below.

Explore the query

Open <http://web-17.challs.olicyber.it/union>. Send a simple numeric input, e.g. `1`, and observe the executed query and the result.

→

Dump the table with a logic injection

Before moving on, try a logic-based injection to reveal the entire contents of `dummy_data`. Does the table contain anything interesting?

' OR '1'='1

→

Console

Invis

OR '1'='1

Query eseguita 91 ms

SELECT * FROM dummy_data WHERE id='1' OR '1'='1'

Result

4, pls stop, 0, example, lollo, 17
3, I'm fake, 0, data, ignoreme, 21
2, Some data, 1, pinki panko, sgrullo, 34
1, dummy values, 3, another_values, lollo, 4

Inject a UNION SELECT

A `UNION SELECT` appends the results of a second `SELECT` to those of the first. The key constraint is that both must return the **same number of columns**. From the previous step you know the original query returns 6 columns.

Send the following payload to verify that the UNION works:

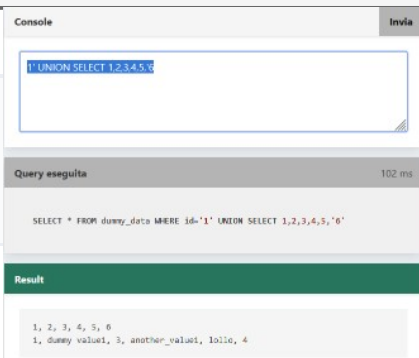
```
1' UNION SELECT 1,2,3,4,5,'6
```

NOTE

This payload avoids the need for a trailing comment: the last value `'6` is left without a closing quote so that the quote appended by the application closes it. Alternatively, you can use `' UNION SELECT 1,2,3,4,5,6 --` with a comment to discard the trailing quote.

What do you see in the result?

→



Fingerprint the database

Different database engines have different system tables and functions. To know which one you are dealing with, extract the version string. Common functions are `version()` (MySQL, PostgreSQL), `@@VERSION` (SQL Server), and `sqlite_version()` (SQLite).

Replace one of the marker values with `version()` in your UNION payload:

```
1' UNION SELECT 1,version(),3,4,5,'6
```

What database engine and version is the backend using?

→

Enumerate tables and columns

MySQL exposes metadata through the `information_schema` schema. Two useful tables are:

- `information_schema.tables` — lists all tables (column `table_name`).
- `information_schema.columns` — lists all columns (columns `table_name`, `column_name`).

To list only the tables in the current database (excluding system tables), filter by `table_schema = DATABASE()`:

```
' UNION SELECT 1,table_name,3,4,5,6 FROM information_schema.tables WHERE  
table_schema=DATABASE() --
```

What tables exist?

→

Now enumerate the columns of the interesting table:

```
' UNION SELECT 1,table_name,3,column_name,5,6 FROM information_schema.columns  
WHERE table_schema=DATABASE() -- -
```

→

Extract the flag

You now know the flag is in the `flag` column of the `real_data` table. Craft the final UNION payload to extract it:

```
' UNION SELECT 1,flag,3,4,5,6 FROM real_data -- -
```

→

Reflection

UNION-based injection is more powerful than logic-based because it lets you read data from **any table** in the database, not just the one the original query targets. However, it requires two conditions: (1) you need to know (or discover) the number of columns, and (2) the query results must be displayed somewhere on the page.

What if the application does not display query results at all, for example if the page only shows “search successful” or “no results”? Could you still use UNION-based injection?

→

Exercise 3 – Basic SQLi

Purpose:	Bypass a login form using a classic logic-based SQL injection.
Suggested tools:	Browser, Burp Suite.
Source:	OliCyber Training – https://training.olicyber.it/challenges#challenge-48

The page (“My secret site”) shows a standard login form with a username and a password field. Invalid credentials produce the message “Incorrect!”. Apply the logic-based injection technique from Exercise 1 to bypass authentication and retrieve the flag.

→

Exercise 4 – Admin’s Secret

Purpose:	Gain admin access by exploiting a SQL injection in the registration flow.
Suggested tools:	Browser, Burp Suite.
Hints:	The login form is not vulnerable. Try registering an account and log in with it: what does the application tell you? Now read the source code of <code>register.php</code> : what SQL statement does registration use, and how is user input handled?
Source:	OliCyber Training – https://training.olicyber.it/challenges#challenge-44

The challenge page (“Admin’s secret”) at <http://adminsecret.challs.olicyber.it/> has two links: *Login* and *Registrazione*. The challenge description also provides the PHP source code for both `login.php` and `register.php`. Read the source code carefully, one page is secure and the other is not.

Start by trying a SQL injection on the login form. Then register a normal account and log in with it. Finally, study the registration source code to find the vulnerability and exploit it.

→

NOTE

SQL injection is not limited to `SELECT` queries or login forms. Any SQL statement that incorporates user input, including `INSERT`, `UPDATE`, and `DELETE`, can be vulnerable. Always test **every** input point in the application, not just the most obvious ones. In this case, reading the source code immediately reveals which page is vulnerable and which is not, a privilege you rarely have in practice, but a useful exercise in understanding the difference between parameterized queries and string concatenation.

Exercise 5 – ToDo

Purpose:	Find and exploit a SQL injection hidden in a session cookie.
Suggested tools:	Browser DevTools, Burp Suite or <code>curl</code> .
Hints:	The injection point is <i>not</i> in any form field. Register an account and look at the cookie the server sets. Read the source code: how is that cookie value used in <code>db.py</code> ? Is it sanitized? What query does it end up in?
Source:	OliCyber Training – https://training.olicyber.it/challenges#challenge-621

The application at <http://todo.challs.olicyber.it> is a to-do list. The landing page is a registration form (username, password, password confirmation). After registering, you are redirected to `/todo` where you can add and toggle tasks. The challenge provides the full source code (a Flask app with SQLite).

Register an account and use the application normally. Then study the source code (`app.py` and `db.py`) to understand how the session is managed and where the flag is stored. Find the vulnerable query and exploit it through the session cookie.

→

NOTE

Injection points are not always in obvious places like form fields or URL parameters. **Any** user-controlled data that reaches a SQL query is a potential injection vector: cookies, HTTP headers (e.g. User-Agent, Referer), and even file names in upload features. A thorough attacker examines every piece of data sent to the server. In this case, most of the queries in `db.py` escape single quotes with `.replace("'", "''")`, but `get_user_from_session` does not, making the cookie the only viable injection point.

Exercise 6 (Optional) – Sn4ck sh3nan1gans

Purpose:	Exploit a SQL injection hidden inside a base64-encoded JSON cookie.
Suggested tools:	Browser DevTools, Burp Suite, CyberChef or command-line <code>base64</code> .
Hints:	Inspect the cookie after logging in. Decode it: what format is it in? What fields does it contain? What happens if you tamper with the <code>ID</code> value?
Source:	OliCyber Training – https://training.olicyber.it/challenges#challenge-39

The application at <http://sn4ck-sh3nan1gans.challs.olicyber.it/> has a login page (“Sign In”) and a registration page (“Sign Up”). Register an account and log in. You land on a welcome page.

Inspect the cookies set by the server: you will find a cookie named `login` whose value is base64-encoded. Decoding it reveals a JSON object containing your numeric user ID, e.g. `{"ID":300}`. This value is used server-side in a SQL query without sanitization.

Tamper with the `ID` field, re-encode the JSON to base64, and replace the cookie. Enumerate the database schema and extract the flag.

→

3 Extras

This section lists additional challenges and resources that are not mandatory for this laboratory. They provide further practice on the topics covered in this lab and can be used for self-study.

OliCyber SQLi Tutorial (continued)

The exercises in this lab covered the first two challenges in the OliCyber SQLi tutorial series (logic-based and UNION-based). The series continues with two more challenges that cover blind injection techniques:

Web 17 – SQLi 3: Error-based / Blind

The third tutorial in the series. The application does not display query results directly, but error messages or differences in server behavior can be used to extract data. This covers the error-based and boolean-based blind techniques.

<https://training.olicyber.it/challenges#challenge-356>

Web 17 – SQLi 4: Time-based

The fourth tutorial in the series. When no output channel is available at all (no results, no errors, no behavioral differences), the attacker can inject time delays (such as `SLEEP()`) and infer data by measuring response times. This is the slowest but most universal blind injection technique.

<https://training.olicyber.it/challenges#challenge-356>

NOTE

The four tutorials form a complete progression from the simplest to the most constrained SQL injection scenarios. Working through all four gives a solid foundation for recognizing and exploiting SQL injection in real-world applications.

Additional challenges

The following challenges cover similar concepts to the exercises above and can serve as extra practice.

SQLiLite

A simple login bypass challenge. The vulnerability is a straightforward logic-based SQL injection on the login form.

<https://play.picoctf.org/practice/challenge/304>

SQL injection – Error based (Root-me)

An error-based SQL injection challenge where you must extract data through verbose database errors. Single quotes are not allowed, so you must find alternative ways to construct your payload.

<https://www.root-me.org/en/Challenges/Web-Server/SQL-injection-Error>

Inception

A SQL injection challenge with heavy input filtering. Many common injection characters and keywords are blocked. Requires creative bypass techniques.

<https://training.olicityber.it/challenges#challenge-134>

PortSwigger Web Security Academy

PortSwigger provides an excellent set of free, guided labs on many web security topics. The SQL injection learning path is directly relevant to this laboratory and covers additional techniques beyond those practiced here:

- **SQL injection:** <https://portswigger.net/web-security/sql-injection>

Includes labs on retrieving hidden data, subverting application logic, UNION attacks, examining the database, blind SQL injection, and second-order SQL injection.